

EC2 Cleanup & Monitoring

Monitor • Review • Clean • Log

PROBLEM

An EC2 server was running low on disk space. I needed to clean it up without accidentally deleting important files or affecting running applications.

HOW I DEBUGGED IT

I started by checking CPU, memory, disk usage, EBS volumes, large files, and running processes before taking any action.

THE SOLUTION

I created a Python-based **EC2 Cleanup & Monitoring script** with:

- **Auto Mode** – performs predefined cleanup tasks
- **Interactive Mode** – reviews actions before applying them
- **JSON Logging** – records cleanup activities

MY APPROACH

Check → Review → Approve → Clean → Log

This keeps the cleanup process controlled and makes it easier to understand what happened.

Has anyone else faced a similar EC2 cleanup problem?

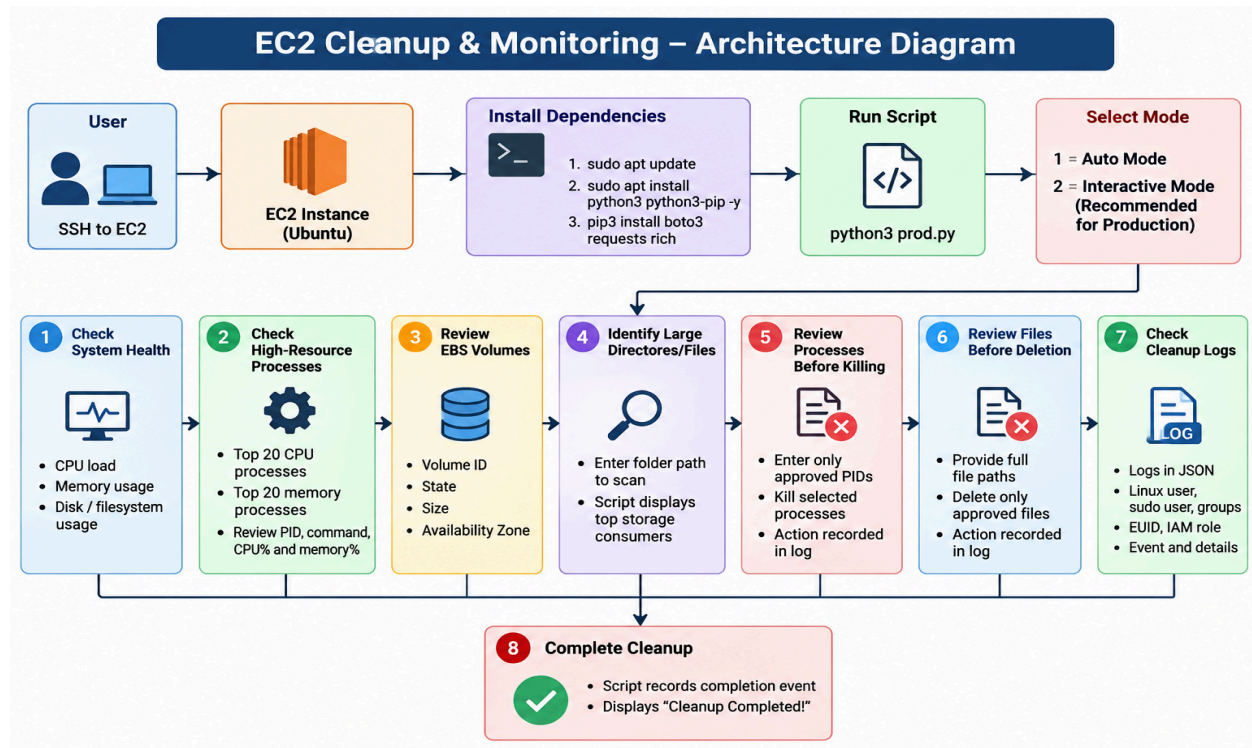
How did you handle it?

How did you fix it?

Blog: <https://vishnusai.hashnode.dev/ec2-cleanup-monitoring-monitor-review-clean-log>

HashnodProfile: <https://hashnode.com/@vishnusaiK>

GitHub: <https://github.com/ksaivishnusaikvs/DevOps-Engineering/tree/main/Linkedin/Post>



Step1: Check System Health Run the cleanup script Select execution mode [./cleanup.py](#)

1 → Auto Mode 2 → Interactive Mode **Interactive Mode is recommended for production** because it asks for approval before critical actions

```
ubuntu@ip-172-31-32-78:~/test$ ./cleanup.py
```

Script Modes:

1. Auto Mode (1)

- The script runs automatically with predefined actions.
- Kills processes, deletes files, scans disks, and logs everything.
- No prompts appear for user input.
- **Use with caution:** only for safe/staging environments or pre-approved process/file lists.

2. Interactive Mode (2) [Default]

- The script pauses and asks for input before every critical action.
- Which PIDs to kill
- Which files to delete
- Folder paths for disk scan
- Requires approval from an authorized user before killing/deleting.
- **Safer for production environments.**

Note:

- Interactive mode is recommended for production.
- Auto mode is faster but may act on processes/files without confirmation.

```
Choose Mode: 1=Auto, 2=Interactive (2): 2
```

SYSTEM MONITOR

```
CPU Load: 0.00, 0.00, 0.00
```

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	823Mi	546Mi	1.0Mi	2.5Gi	2.7Gi
Swap:	0B	0B	0B			

Filesystem	Size	Used	Avail	Use%	Mounted on
/dev/root	97G	9.2G	88G	10%	/
/dev/xvda15	105M	6.1M	99M	6%	/boot/efi

Step2: Check High-Resource Processes

Next, identify the processes consuming the most system resources. The script displays the **Top 20 CPU-consuming processes** and **Top 20 memory-consuming processes**, including the **PID**, **command**, **CPU percentage**, and **memory percentage**

TOP 20 CPU PROCESSES				
PID	PPID	Command	%CPU	%MEM
6356	3098	python3	0.7	0.4
381	1	amazon-cloudwat	0.6	3.0
793	1	mysqld	0.4	9.8
1447	1403	promtail	0.3	1.4
1506	1472	grafana	0.2	5.9
1448	1407	loki	0.1	1.7
1	0	systemd	0.0	0.3
2	0	kthreadd	0.0	0.0
3	2	pool_workqueue_	0.0	0.0
4	2	kworker/R-rcu_g	0.0	0.0
5	2	kworker/R-rcu_p	0.0	0.0
6	2	kworker/R-slub_	0.0	0.0
7	2	kworker/R-netns	0.0	0.0
9	2	kworker/0:0H-kb	0.0	0.0
12	2	kworker/R-mm_pe	0.0	0.0
13	2	rcu_tasks_rude_	0.0	0.0
14	2	rcu_tasks_trace	0.0	0.0
15	2	ksoftirqd/0	0.0	0.0
16	2	rcu_sched	0.0	0.0
17	2	migration/0	0.0	0.0

TOP 20 MEMORY PROCESSES				
PID	PPID	Command	%CPU	%MEM
793	1	mysqld	0.4	9.8
1506	1472	grafana	0.2	5.9
381	1	amazon-cloudwat	0.6	3.0
135	1	systemd-journal	0.0	2.6
734	1	dockerd	0.0	2.0
1448	1407	loki	0.1	1.7
1447	1403	promtail	0.3	1.4
435	1	containerd	0.0	1.2
423	1	snaped	0.0	0.9
738	382	ssm-agent-worke	0.0	0.7
173	1	multipathd	0.0	0.6
570	1	unattended-upgr	0.0	0.5
5037	1	packagekitd	0.0	0.5
382	1	amazon-ssm-agen	0.0	0.4
411	1	networkd-dispat	0.0	0.4
6356	3098	python3	0.9	0.4
1407	1	containerd-shim	0.0	0.3
1403	1	containerd-shim	0.0	0.3

Step3: Review EBS Volumes

After checking the EC2 instance, review the attached EBS volumes. The script displays important information such as the Volume ID, volume state, size, and Availability Zone.

Next, **identify large files/directories and review processes or files** before killing processes or deleting unnecessary files.

Enter the required **folder path**, and the script scans the path and displays the **top storage-consuming files and directories**

Review Processes – Approve PIDs before killing them.

Review Files – Approve files before deleting them.

```
===== EBS VOLUMES (Top 20) =====
----- DescribeVolumes -----
+-----+-----+-----+-----+
| AZ      | ID              | Size | State |
+-----+-----+-----+-----+
| us-east-1c | vol-0798aa3dc96dbbc80 | 100 | in-use |
+-----+-----+-----+-----+

Enter PIDs to kill from CPU list (space separated, blank to skip) O:
Enter PIDs to kill from Memory list (space separated, blank to skip) O:
Enter folder path to scan (/):

===== DISK USAGE (/) =====
6.4G /
2.6G /usr
1.5G /snap
1.4G /var
1.1G /usr/lib
1010M /var/lib
641M /usr/bin
618M /var/lib/snapd
496M /snap/core22
426M /opt/aws/amazon-cloudwatch-agent
426M /opt/aws
426M /opt
425M /opt/aws/amazon-cloudwatch-agent/bin
419M /snap/core20
368M /usr/share
349M /home/ubuntu
349M /home
312M /var/lib/snapd/snaps
311M /usr/src
306M /var/lib/snapd/seed/snaps

Enter full file paths to delete (space separated, blank to skip) O:

[+] Cleanup Completed!
```

Step4: Check Cleanup Logs

All important cleanup activities are recorded in JSON log format. The logs capture details such as the Linux user, sudo user, Linux groups, EUID, IAM role, event, timestamp, and action details, providing an audit trail of the cleanup activity

Check logs :

```
ubuntu@ip-172-31-32-78:~/logs$ ls
prod_cleanup.log  prod_cleanup_shell.log
ubuntu@ip-172-31-32-78:~/logs$ cat prod_cleanup.log
{"ts": "2025-09-11T12:39:19Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "<?xml version='1.0' encoding='iso-8859-1'?'><DOCTYPE html PUBLIC '-//W3C//DTD XHTML 1.0 Transitional//EN'\n\t'http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd'\n>\n<html xmlns='http://www.w3.org/1999/xhtml'\n\n xml:lang='en'\n\n lang='en'\n\n <head>\n\n <title>401 - Unauthorized</title>\n\n </head>\n\n <body>\n\n <h1>401 - Unauthorized</h1>\n\n </body>\n\n </html>"; "event": "monitor", "detail": "System stats displayed"}
```

```
{"ts": "2025-09-11T13:33:47Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "<?xml version='1.0' encoding='iso-8859-1'?'><DOCTYPE html PUBLIC '-//W3C//DTD XHTML 1.0 Transitional//EN'\n\t'http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd'\n>\n<html xmlns='http://www.w3.org/1999/xhtml'\n\n xml:lang='en'\n\n lang='en'\n\n <head>\n\n <title>401 - Unauthorized</title>\n\n </head>\n\n <body>\n\n <h1>401 - Unauthorized</h1>\n\n </body>\n\n </html>"; "event": "finish", "detail": "Cleanup completed"}
{"ts": "2025-09-11T13:45:24Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "none", "event": "monitor", "detail": "System stats displayed"}
{"ts": "2025-09-11T13:45:24Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "none", "event": "monitor", "detail": "Top CPU and memory processes displayed"}
{"ts": "2025-09-11T13:45:25Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "none", "event": "ebs", "detail": "EBS volumes listed"}
{"ts": "2025-09-11T13:46:12Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "none", "event": "disk", "detail": "Top storage usage scanned in path /"}
{"ts": "2025-09-11T13:46:28Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "none", "event": "finish", "detail": "Cleanup completed"}
{"ts": "2025-09-11T14:37:59Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "none", "event": "monitor", "detail": "System stats displayed"}
{"ts": "2025-09-11T14:37:59Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "none", "event": "monitor", "detail": "Top CPU and memory processes displayed"}
{"ts": "2025-09-11T14:37:59Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "none", "event": "ebs", "detail": "EBS volumes listed"}
{"ts": "2025-09-11T14:38:19Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "none", "event": "disk", "detail": "Top storage usage scanned in path /"}
{"ts": "2025-09-11T14:38:23Z", "linux_user": "ubuntu", "sudo_user": "ubuntu", "linux_groups": "adm,dialout,cdrom,floppy,sudo,audio,dip,video,plugdev,netdev,lxd,docker,ubuntu", "euid": 1000, "iam_role": "none", "event": "finish", "detail": "Cleanup completed"}
ubuntu@ip-172-31-32-78:~/logs$
```

Python script

```
#!/usr/bin/env python3
```

```
"""
```

```
Production Cleanup & Monitoring Script
```

```
"""
```

```
import os
import json
import subprocess
import datetime
import getpass
import grp
from pathlib import Path
from rich.console import Console
from rich.prompt import Prompt
from rich.table import Table
```

```
console = Console()
```

```
# Log file setup: try /var/log first, else fallback to ~/logs
```

```
DEFAULT_LOG = Path("/var/log/prod_cleanup.log")
```

```
FALLBACK_LOG = Path.home() / "logs" / "prod_cleanup.log"
```

```
LOG_FILE = DEFAULT_LOG if os.access(DEFAULT_LOG.parent, os.W_OK) else FALLBACK_LOG
```

```
def show_description():
```

```
    console.print("""
```

```
[bold cyan]=====[/bold cyan]
```

```
[bold yellow]Script Modes:[/bold yellow]
```

```
[bold green]1. Auto Mode (1)[/bold green]
```

- The script runs automatically with predefined actions.
- Kills processes, deletes files, scans disks, and logs everything.
- No prompts appear for user input.
- [red]Use with caution:[/red] only for safe/staging environments or pre-approved process/file lists.

```
[bold green]2. Interactive Mode (2) [Default][/bold green]
```

- The script pauses and asks for input before every critical action.
- Which PIDs to kill
- Which files to delete
- Folder paths for disk scan
- Requires approval from an authorized user before killing/deleting.
- [yellow]Safer for production environments.[/yellow]

```
[bold magenta]Note:[/bold magenta]
```

- Interactive mode is recommended for production.
- Auto mode is faster but may act on processes/files without confirmation.

```
[bold cyan]=====[/bold cyan]
""")
```

```
def log_event(event, detail):
    """Log event in JSON format with safe IAM role detection"""
    ts = datetime.datetime.utcnow().strftime("%Y-%m-%dT%H:%M:%SZ")
    linux_user = getpass.getuser()
    sudo_user = os.environ.get("SUDO_USER", linux_user)
    linux_groups = ",".join(grp.getgrgid(g).gr_name for g in os.getgroups())
    euid = os.geteuid()

    # Detect IAM role safely
    try:
        role_name = subprocess.check_output(
            ["curl", "-s", "http://169.254.169.254/latest/meta-data/iam/security-credentials/"],
            text=True, timeout=2
        ).strip()
        if role_name and "Unauthorized" not in role_name:
            iam_role = role_name
        else:
            iam_role = "none"
    except Exception:
        iam_role = "none"

    log_data = {
        "ts": ts,
        "linux_user": linux_user,
        "sudo_user": sudo_user,
        "linux_groups": linux_groups,
        "euid": euid,
        "iam_role": iam_role,
        "event": event,
        "detail": detail
    }
    LOG_FILE.parent.mkdir(parents=True, exist_ok=True)
    with LOG_FILE.open("a") as f:
        f.write(json.dumps(log_data) + "\n")

def run_cmd(cmd):
    try:
        return subprocess.check_output(cmd, shell=True, text=True)
    except subprocess.CalledProcessError:
        return ""

def system_monitor():
    console.print("\n[bold cyan]===== SYSTEM MONITOR =====[/bold cyan]")
```

```

uptime_output = run_cmd("uptime | awk -F'load average:' '{print $2}").strip()
console.print(f"[bold yellow]CPU Load:[/bold yellow] {uptime_output}")
console.print(run_cmd("free -h"))
console.print(run_cmd("df -h | grep -E '^/dev|Filesystem'"))
log_event("monitor", "System stats displayed")

```

```
def show_top_processes():
```

```

    # CPU
    console.print("\n[bold cyan]===== TOP 20 CPU PROCESSES =====[/bold cyan]")
    cpu_output = run_cmd("ps -eo pid,ppid,comm,%cpu,%mem --sort=-%cpu | head -n 21").splitlines()
    table_cpu = Table(show_header=True, header_style="bold magenta")
    table_cpu.add_column("PID", justify="right")
    table_cpu.add_column("PPID", justify="right")
    table_cpu.add_column("Command", overflow="fold")
    table_cpu.add_column("%CPU", justify="right")
    table_cpu.add_column("%MEM", justify="right")

    for line in cpu_output[1:]:
        parts = line.split(None, 4)
        if len(parts) == 5:
            table_cpu.add_row(*parts)
    console.print(table_cpu)

    # Memory
    console.print("\n[bold cyan]===== TOP 20 MEMORY PROCESSES =====[/bold cyan]")
    mem_output = run_cmd("ps -eo pid,ppid,comm,%cpu,%mem --sort=-%mem | head -n 21").splitlines()
    table_mem = Table(show_header=True, header_style="bold magenta")
    table_mem.add_column("PID", justify="right")
    table_mem.add_column("PPID", justify="right")
    table_mem.add_column("Command", overflow="fold")
    table_mem.add_column("%CPU", justify="right")
    table_mem.add_column("%MEM", justify="right")

    for line in mem_output[1:]:
        parts = line.split(None, 4)
        if len(parts) == 5:
            table_mem.add_row(*parts)
    console.print(table_mem)

    log_event("monitor", "Top CPU and memory processes displayed")
    return cpu_output, mem_output

```

```
def kill_processes(process_list, label):
```

```

    pids = Prompt.ask(f"Enter PIDs to kill from {label} list (space separated, blank to skip)", default="")
    if pids:
        for pid in pids.split():
            try:

```

```

        os.kill(int(pid), 9)
        console.print(f"[green] Killed process {pid}[/green]")
        log_event("kill", f"Process {pid} killed from {label}")
    except Exception:
        console.print(f"[red] Failed to kill {pid}[/red]")

```

```

def list_ebs_volumes():
    console.print("\n[bold cyan]===== EBS VOLUMES (Top 20) =====[/bold cyan]")
    if run_cmd("which aws"):
        volumes = run_cmd(
            "aws ec2 describe-volumes --query"
            "\"Volumes[*].{ID:Volumeld,State:State,Size:Size,AZ:AvailabilityZone}\" --output table | head -n 22"
        )
        console.print(volumes if volumes else "[yellow]No volumes found[/yellow]")
        log_event("ebs", "EBS volumes listed")
    else:
        console.print("[yellow]AWS CLI not installed. Skipping EBS volume listing.[/yellow]")
        log_event("ebs", "AWS CLI not installed")

```

```

def disk_usage():
    path = Prompt.ask("Enter folder path to scan", default="/")
    console.print(f"\n[bold cyan]===== DISK USAGE ({path}) =====[/bold cyan]")
    usage = run_cmd(f"du -ah {path} 2>/dev/null | sort -rh | head -n 20")
    console.print(usage)
    log_event("disk", f"Top storage usage scanned in path {path}")

```

```

def delete_files():
    files = Prompt.ask("Enter full file paths to delete (space separated, blank to skip)", default="")
    if files:
        for f in files.split():
            try:
                os.remove(f)
                console.print(f"[green] Deleted {f}[/green]")
                log_event("delete", f"File {f} deleted")
            except FileNotFoundError:
                console.print(f"[yellow] File {f} not found[/yellow]")

```

```

def main():
    show_description()
    mode = Prompt.ask("Choose Mode: [green]1=Auto[/green], [yellow]2=Interactive[/yellow]", default="2")

    system_monitor()
    cpu_list, mem_list = show_top_processes()
    list_ebs_volumes()

```

```
if mode == "2":  
    kill_processes(cpu_list, "CPU")  
    kill_processes(mem_list, "Memory")  
    disk_usage()  
    delete_files()  
  
log_event("finish", "Cleanup completed")  
console.print("\n[bold green] Cleanup Completed![/bold green]")
```

```
if __name__ == "__main__":  
    main()
```

If run the script

